README

Exercise 2 Gesture Classification Output Sketch

Real-Time Gesture Visualization Using OSC Output from Wekinator and Processing

`GestureOutput.pde`

Contents

1 Purpose of This Report **4**

2 Big Picture Explanation **4**

3 What This Sketch Does **5**

4 Input and Output of the Sketch **5**

4.1 Input 5

4.2 Output 5

5 Relationship with the Full Project **6**

6 Correct Wekinator Settings for This Sketch **6**

7 Why the Output Sketch Is Separate from the Input Sketch **7**

8 Complete Code Being Explained **7**

9 Line-by-Line and Block-by-Block Explanation **11**

9.1 Library Imports 11

9.2 Header Comment 11

9.3 OSC Object 12

9.4 OSC Receiving Settings 12

9.5 Gesture Labels 13

9.6 Current Prediction Variables 13

10 The `setup()` Function **14**

10.1 Starting OSC Listening 14

10.2 Text Settings 14

10.3 Console Debug Messages 15

11 The `draw()` Function **15**

12 The `drawBackground()` Function **16**

13 The `drawMainText()` Function **17**

14 The `drawDebugInfo()` Function **18**

14.1 Message Indicator Dot 19

15 The `oscEvent()` Function **20**

15.1 Check the OSC Address . . . 20
15.2 Check That the Message Has a Value . . . 21
15.3 Read the Class Value . . . 21
15.4 Update Debug Variables . . . 22

16 The `classToGesture()` Function . . . 22

17 The `keyPressed()` Function . . . 23

18 Complete Program Flow . . . 24

19 Correct Running Order . . . 24
19.1 During Training . . . 24
19.2 During Testing and Demonstration . . . 24

20 Example Use Case . . . 25

21 How This Code Fulfills the Exercise 2 Requirements . . . 25

22 Strengths of This Output Sketch . . . 26
22.1 Clear Visualization . . . 26
22.2 Simple Class Mapping . . . 26
22.3 Correct OSC Setup . . . 26
22.4 Debugging Support . . . 27
22.5 Safe Message Handling . . . 27

23 Limitations of This Sketch . . . 27
23.1 It Depends on Wekinator . . . 27
23.2 It Only Handles Two Named Classes by Default . . . 27
23.3 It Does Not Show Camera Input . . . 27
23.4 It Cannot Fix Bad Training . . . 27

24 Debugging Checklist . . . 27
24.1 If the Output Sketch Stays on "Waiting for Wekinator..." . . . 27
24.2 If Message Count Stays at 0 . . . 28
24.3 If the Message Count Increases but the Gesture Is Wrong . . . 28
24.4 If the Raw Value Is Not 1.0 or 2.0 . . . 28

25 How to Extend This Code for More Classes . . . 29

26 Explanation of OSC in This Sketch 29

27 Explanation of the Classifier Output 30

28 Why This Output Is Useful for the Project 30

29 Recommended Final Project Description for This Sketch 31

30 Final Conclusion 31

1. Purpose of This Report

This README explains the `GestureOutput.pde` Processing sketch completely and comprehensively.

The report has three main goals:

1. Explain the **Processing code itself** as a standalone program.
2. Explain the **functionality of each important line and block** of code.
3. Explain how this sketch fulfills the output/visualization requirements of **Exercise 2 — Gesture Classification**.

This sketch is the **output side** of the gesture classification system. It does not use the webcam, does not extract image features, and does not train a machine learning model. Its job is to receive the class predicted by Wekinator and visualize that class on screen.

2. Big Picture Explanation

This sketch waits for messages from Wekinator. When Wekinator recognizes a gesture, it sends a classification output such as:

```
1.0
```

or:

```
2.0
```

The Processing output sketch receives this value and converts it into a meaningful gesture label.

For the current project:

```
Class 1 → Open Hand
Class 2 → Closed Fist
```

The complete output pipeline is:

```
Wekinator predicts a gesture class
        ↓
Wekinator sends OSC message /wek/outputs
        ↓
Processing output sketch receives the value
        ↓
The value is rounded to an integer class number
        ↓
The class number is converted into a gesture name
        ↓
The gesture name is displayed on screen
```

This sketch is the final visual feedback stage of the interactive machine learning system.

3. What This Sketch Does

The sketch performs these tasks:

1. Imports the OSC libraries.
2. Opens an OSC receiving port.
3. Waits for Wekinator output messages.
4. Checks that the received message has the correct address: `/wek/outputs`.
5. Reads the first value of the OSC message.
6. Converts the value into a class number.
7. Maps the class number to a gesture label.
8. Displays the recognized gesture on screen.
9. Changes the background color according to the class.
10. Displays debugging information such as port, OSC address, message count, and last received value.
11. Provides a reset key to return the display to the waiting state.

4. Input and Output of the Sketch

4.1. Input

The input of this sketch is not the webcam. The input is an OSC message from Wekinator.

The sketch listens for:

```
OSC address: /wek/outputs
Port: 12000
```

Wekinator sends one numerical value:

```
1.0 → Class 1
2.0 → Class 2
```

The value is a classification label. It is not an image and it is not a feature vector.

4.2. Output

The output of this sketch is visual feedback in the Processing window.

It displays:

- The received class number
- The recognized gesture name
- A background color for the current class

- Debug information
- A small message-received indicator dot

For the current configuration:

Wekinator Output	Meaning	Processing Display
`1.0`	Class 1	`Open Hand`
`2.0`	Class 2	`Closed Fist`
Other value	Unknown class	`Unknown Class`

5. Relationship with the Full Project

Exercise 2 has two Processing sketches and one Wekinator project.

The full system is:

```
Input Processing sketch
(webcam, crop, downsample, pixel features)
        ↓
OSC /wek/inputs on port 6448
        ↓
Wekinator
(KNN classifier, 2 gesture classes)
        ↓
OSC /wek/outputs on port 12000
        ↓
Output Processing sketch
(display recognized gesture label)
```

This `GestureOutput.pde` sketch corresponds to the last part of the system:

```
Wekinator → Processing output visualization
```

It confirms whether the trained classifier is working by showing the predicted class in real time.

6. Correct Wekinator Settings for This Sketch

For this output sketch to work, Wekinator must be configured correctly.

The required Wekinator output settings are:

Setting	Correct Value
Output OSC message	`/wek/outputs`
Output host	`127.0.0.1` or `localhost`
Output port	`12000`
Number of outputs	`1`
Output type	Classification
Number of classes	`2`

The input side of the Wekinator project should also match the input sketch:

Setting	Correct Value for Current Project
Input port	`6448`
Input OSC message	`/wek/inputs`
Number of inputs	`2500`

The `2500` input count comes from the input sketch configuration:

```
50 × 50 grayscale = 2500 input features
```

7. Why the Output Sketch Is Separate from the Input Sketch

The project uses two Processing sketches because each sketch has a different responsibility.

Sketch	Responsibility
Input sketch	Captures webcam image, extracts features, sends data to Wekinator
Output sketch	Receives Wekinator prediction and displays the recognized gesture

This separation makes the system easier to understand and debug.

The input sketch answers:

```
What does the camera currently see?
```

The output sketch answers:

```
What class did Wekinator predict?
```

8. Complete Code Being Explained

```
import oscP5.*;
import netP5.*;

// ============================================================
// Exercise 2 -- Corrected Gesture Classification Output Sketch
// Wekinator → /wek/outputs → Processing text label display
// ============================================================
//
// This is the OUTPUT sketch for Exercise 2.
// It receives the predicted gesture class from Wekinator and displays
// the corresponding gesture name on the screen.
//
// Wekinator output settings:
//   Output OSC message: /wek/outputs
//   Output host: 127.0.0.1 or localhost
//   Output port: 12000
//   Number of outputs: 1
//   Output type: Classification
//   Number of classes: 2
//
// Class mapping in this sketch:
//   Class 1 -> Open Hand
//   Class 2 -> Closed Fist
//
// Change class1Gesture and class2Gesture if your gestures are different.
// ============================================================

OscP5 oscP5;

// ----------------------------
// OSC receiving settings
// ----------------------------
int inPort = 12000;
String oscAddress = "/wek/outputs";

// ----------------------------
// Gesture labels
// ----------------------------
String class1Gesture = "Open Hand";
String class2Gesture = "Closed Fist";

// ----------------------------
// Current prediction values
// ----------------------------
int currentClass = 0;
String currentGesture = "Waiting for Wekinator...";
float lastRawValue = 0.0;
int messageCount = 0;
int lastMessageTime = 0;

void setup() {
  size(800, 450);

  // Listen for Wekinator output messages on port 12000
```

```
  oscP5 = new OscP5(this, inPort);

  textAlign(CENTER, CENTER);
  textFont(createFont("Arial", 32));

  println("====================================");
  println("Exercise 2 Gesture Output Sketch");
  println("Listening on port: " + inPort);
  println("Expected OSC address: " + oscAddress);
  println("Class 1: " + class1Gesture);
  println("Class 2: " + class2Gesture);
  println("====================================");
}

void draw() {
  drawBackground();
  drawMainText();
  drawDebugInfo();
}

// ============================================================
// Background changes according to the current predicted class
// ============================================================

void drawBackground() {
  if (currentClass == 1) {
    background(210, 60, 60);      // Class 1 background
  } else if (currentClass == 2) {
    background(60, 90, 210);      // Class 2 background
  } else {
    background(25);               // Waiting / unknown background
  }
}

// ============================================================
// Main text display
// ============================================================

void drawMainText() {
  fill(255);

  textSize(26);
  text("Received gesture class:", width / 2, 80);

  textSize(64);
  text(currentClass, width / 2, 155);

  textSize(26);
  text("Recognized gesture:", width / 2, 245);

  textSize(56);
  text(currentGesture, width / 2, 320);
}
```

```
// ============================================================
// Small debugging information
// ============================================================

void drawDebugInfo() {
  fill(0, 150);
  noStroke();
  rect(20, 20, 260, 100, 8);

  fill(255);
  textAlign(LEFT, TOP);
  textSize(14);
  text("OSC port: " + inPort, 35, 35);
  text("OSC address: " + oscAddress, 35, 58);
  text("Messages: " + messageCount, 35, 81);
  text("Last raw value: " + nf(lastRawValue, 1, 3), 35, 104);
  textAlign(CENTER, CENTER);

  // Green dot flashes when a message is received
  if (millis() - lastMessageTime < 250) {
    fill(0, 255, 0);
  } else {
    fill(120);
  }
  ellipse(width - 40, 40, 22, 22);
}

// ============================================================
// Receive OSC messages from Wekinator
// ============================================================

void oscEvent(OscMessage msg) {
  // Only use the correct Wekinator output message
  if (!msg.checkAddrPattern(oscAddress)) {
    println("Ignored message: " + msg.addrPattern());
    return;
  }

  // Wekinator classification output should contain one value:
  // 1.0 for Class 1, 2.0 for Class 2, etc.
  if (msg.arguments().length < 1) {
    println("Received /wek/outputs with no values.");
    return;
  }

  lastRawValue = msg.get(0).floatValue();
  currentClass = round(lastRawValue);
  currentGesture = classToGesture(currentClass);

  messageCount++;
  lastMessageTime = millis();

  println("Received: " + lastRawValue + " -> class " + currentClass + " -> " +
    currentGesture);
```

```
}

// ============================================================
// Convert class number to gesture label
// ============================================================

String classToGesture(int c) {
  switch(c) {
    case 1:
      return class1Gesture;
    case 2:
      return class2Gesture;
    default:
      return "Unknown Class";
  }
}

// ============================================================
// Optional reset
// ============================================================

void keyPressed() {
  if (key == 'r' || key == 'R') {
    currentClass = 0;
    currentGesture = "Waiting for Wekinator...";
    lastRawValue = 0.0;
    println("Reset output display.");
  }
}
```

9. Line-by-Line and Block-by-Block Explanation

9.1. Library Imports

```
import oscP5.*;
```

This imports the oscP5 library. The oscP5 library allows Processing to receive and send OSC messages.

In this sketch, oscP5 is mainly used to **receive** messages from Wekinator.

```
import netP5.*;
```

This imports the netP5 library. netP5 is often used together with oscP5 for OSC networking. In this output sketch, it is included for compatibility with OSC communication, even though the sketch mainly listens rather than sends.

9.2. Header Comment

```
// ============================================================
// Exercise 2 -- Corrected Gesture Classification Output Sketch
// Wekinator → /wek/outputs → Processing text label display
```

```
// ============================================================
```

This comment describes the sketch. It explains that this file is the corrected output sketch for Exercise 2.

```
// This is the OUTPUT sketch for Exercise 2.
```

This explains the role of the sketch. It is not the input sketch.

```
// It receives the predicted gesture class from Wekinator and displays
// the corresponding gesture name on the screen.
```

This explains the main function: receive prediction, show gesture name.

The comments also list the correct Wekinator settings:

```
Output OSC message: /wek/outputs
Output host: 127.0.0.1 or localhost
Output port: 12000
Number of outputs: 1
Output type: Classification
Number of classes: 2
```

This is very useful because the code will not work unless Wekinator uses matching settings.

9.3. OSC Object

```
OscP5 oscP5;
```

This declares an OSC object called `oscP5`.

This object is responsible for opening the OSC port and receiving messages from Wekinator.

9.4. OSC Receiving Settings

```
int inPort = 12000;
```

This sets the port where Processing listens for Wekinator output.

Wekinator must send its output to this same port.

So Wekinator output port must be:

```
12000
```

```
String oscAddress = "/wek/outputs";
```

This stores the OSC address that the sketch expects.

Wekinator must send output messages using this address:

```
/wek/outputs
```

If the message address is different, the sketch ignores the message.

9.5. Gesture Labels

```
String class1Gesture = "Open Hand";
```

This defines the label for Class 1.

If Wekinator sends class value `1.0`, the sketch displays:

```
Open Hand
```

```
String class2Gesture = "Closed Fist";
```

This defines the label for Class 2.

If Wekinator sends class value `2.0`, the sketch displays:

```
Closed Fist
```

These labels can be changed if the project uses different gestures.

For example:

```
String class1Gesture = "Thumbs Up";
String class2Gesture = "Flat Palm";
```

9.6. Current Prediction Variables

```
int currentClass = 0;
```

This stores the current predicted class number.

At the beginning, it is `0`, which means no valid class has been received yet.

```
String currentGesture = "Waiting for Wekinator...";
```

This stores the current gesture label shown on screen.

At the beginning, the sketch displays a waiting message because Wekinator has not sent any prediction yet.

```
float lastRawValue = 0.0;
```

This stores the exact raw value received from Wekinator.

For classification, Wekinator usually sends values like:

```
1.0
2.0
```

The sketch stores this raw value for debugging.

```
int messageCount = 0;
```

This counts how many valid OSC messages the sketch has received from Wekinator.

It helps confirm that communication is working.

```
int lastMessageTime = 0;
```

This stores the time when the last message was received.

It is used to make the small indicator dot flash green when a message arrives.

10. The `setup()` Function

The `setup()` function runs once when the Processing sketch starts.

```
void setup() {
```

This begins the setup function.

```
size(800, 450);
```

This creates a Processing window that is 800 pixels wide and 450 pixels tall.

10.1. Starting OSC Listening

```
oscP5 = new OscP5(this, inPort);
```

This starts oscP5 and tells Processing to listen on port `12000`.

This is the most important line for receiving Wekinator output.

It means:

```
Processing is waiting for OSC messages on port 12000
```

Wekinator must send to this port.

10.2. Text Settings

```
textAlign(CENTER, CENTER);
```

This sets text alignment so that text is centered both horizontally and vertically around its coordinate.

For example:

```
text(currentGesture, width / 2, 320);
```

will place the text around the center of the given point.

```
textFont(createFont("Arial", 32));
```

This creates and sets the font used by the sketch. The font is Arial with a default size of 32.

10.3. Console Debug Messages

```
println("====================================");
```

This prints a separator line in the Processing console.

```
println("Exercise 2 Gesture Output Sketch");
```

This prints the sketch name.

```
println("Listening on port: " + inPort);
```

This prints the OSC listening port.

```
println("Expected OSC address: " + oscAddress);
```

This prints the OSC address the sketch expects.

```
println("Class 1: " + class1Gesture);
```

This prints the label for Class 1.

```
println("Class 2: " + class2Gesture);
```

This prints the label for Class 2.

```
println("====================================");
```

This prints another separator line.

These console messages are useful because they confirm that the sketch started correctly.

```
}
```

This closes the `setup()` function.

11. The `draw()` Function

The `draw()` function runs continuously while the sketch is open.

```
void draw() {
```

This begins the draw loop.

```
drawBackground();
```

This calls the function that draws the background color according to the current class.

```
drawMainText();
```

This calls the function that displays the class number and gesture label.

```
drawDebugInfo();
```

This calls the function that displays technical debugging information.

```
}
```

This closes the draw loop.

The draw function does not receive OSC directly. It only displays whatever the latest received class is.

The OSC receiving happens in the `oscEvent()` function.

12. The `drawBackground()` Function

This function changes the background based on the current predicted class.

```
void drawBackground() {
```

This starts the function.

```
if (currentClass == 1) {
```

This checks if the predicted class is Class 1.

```
background(210, 60, 60);
```

This sets the background to a red color for Class 1.

In this project, Class 1 means:

```
Open Hand
```

```
} else if (currentClass == 2) {
```

This checks if the predicted class is Class 2.

```
background(60, 90, 210);
```

This sets the background to a blue color for Class 2.

In this project, Class 2 means:

```
Closed Fist
```

```
} else {
```

This runs if the class is neither 1 nor 2.

```
background(25);
```

This sets a dark background. It is used while waiting or when an unknown class is received.

```
}
```

This closes the class condition.

```
}
```

This closes the function.

13. The `drawMainText()` Function

This function draws the main user-facing result.

```
void drawMainText() {
```

This starts the function.

```
fill(255);
```

This sets the text color to white.

```
textSize(26);
```

This sets the text size to 26.

```
text("Received gesture class:", width / 2, 80);
```

This displays a label at the top of the screen.

`width / 2` places it horizontally in the center.

```
textSize(64);
```

This makes the next text larger.

```
text(currentClass, width / 2, 155);
```

This displays the current class number.

For example:

```
1
```

or:

```
2
```

```
textSize(26);
```

This changes the text size back to 26.

```
text("Recognized gesture:", width / 2, 245);
```

This displays another label before the gesture name.

```
textSize(56);
```

This makes the gesture name large and easy to read.

```
text(currentGesture, width / 2, 320);
```

This displays the gesture name, such as:

```
Open Hand
```

or:

```
Closed Fist
```

```
}
```

This closes the function.

14. The `drawDebugInfo()` Function

This function draws a small technical panel in the corner.

```
void drawDebugInfo() {
```

This starts the function.

```
fill(0, 150);
```

This sets a semi-transparent black fill color.

```
noStroke();
```

This removes the border around shapes.

```
rect(20, 20, 260, 100, 8);
```

This draws a rounded rectangle for the debug panel.

The values mean:

```
x = 20
y = 20
width = 260
height = 100
corner radius = 8
```

```
fill(255);
```

This sets the text color to white.

```
textAlign(LEFT, TOP);
```

This temporarily changes text alignment so the debug text starts from the left/top position.

```
textSize(14);
```

This sets the debug text size to 14.

```
text("OSC port: " + inPort, 35, 35);
```

This displays the OSC listening port.

```
text("OSC address: " + oscAddress, 35, 58);
```

This displays the OSC address that the sketch expects.

```
text("Messages: " + messageCount, 35, 81);
```

This displays how many valid Wekinator messages have been received.

If this number stays at `0`, Wekinator is not sending correctly or the output sketch is not receiving correctly.

```
text("Last raw value: " + nf(lastRawValue, 1, 3), 35, 104);
```

This displays the last raw value received from Wekinator.

The function `nf(lastRawValue, 1, 3)` formats the number with three decimal places.

For example:

```
1.000
```

or:

```
2.000
```

```
textAlign(CENTER, CENTER);
```

This restores the text alignment back to center, which is needed for the main screen text.

14.1. Message Indicator Dot

```
if (millis() - lastMessageTime < 250) {
```

This checks whether a message was received recently.

`millis()` gives the number of milliseconds since the sketch started.

If the last message arrived less than 250 milliseconds ago, the dot becomes green.

```
fill(0, 255, 0);
```

This sets the dot color to green.

```
} else {
```

This runs when no message has been received recently.

```
fill(120);
```

This sets the dot color to gray.

```
}
```

This closes the time check.

```
ellipse(width - 40, 40, 22, 22);
```

This draws the indicator dot near the top-right corner of the screen.

If Wekinator is sending continuously, this dot should flash or stay green often.

```
}
```

This closes the debug function.

15. The `oscEvent()` Function

This is the most important function in the output sketch.

It automatically runs whenever an OSC message arrives.

```
void oscEvent(OscMessage msg) {
```

This starts the OSC event handler. The received OSC message is stored in the variable `msg`.

15.1. Check the OSC Address

```
if (!msg.checkAddrPattern(oscAddress)) {
```

This checks whether the received message address is not equal to `/wek/outputs`.

The `!` means "not".

So this line means:

```
If the message is NOT /wek/outputs, ignore it.
```

```
println("Ignored message: " + msg.addrPattern());
```

This prints the ignored message address to the console.

This is useful for debugging because it shows if another OSC message is arriving.

```
return;
```

This stops the function immediately. The sketch does not process messages with the wrong address.

```
}
```

This closes the address check.

15.2. Check That the Message Has a Value

```
if (msg.arguments().length < 1) {
```

This checks whether the message contains no values.

Wekinator should send one output value, so an empty message is invalid.

```
println("Received /wek/outputs with no values.");
```

This prints an error message if the message has no data.

```
return;
```

This stops the function so the sketch does not crash or use missing data.

```
}
```

This closes the empty-message check.

15.3. Read the Class Value

```
lastRawValue = msg.get(0).floatValue();
```

This reads the first value in the OSC message and stores it as a float.

For Wekinator classification, this value is usually:

```
1.0
```

or:

```
2.0
```

```
currentClass = round(lastRawValue);
```

This rounds the raw float value into an integer.

For example:

```
1.0 → 1
2.0 → 2
```

This is important because the rest of the program uses integer class numbers.

```
currentGesture = classToGesture(currentClass);
```

This calls the `classToGesture()` function and converts the class number into a readable gesture label.

For example:

```
1 → Open Hand
2 → Closed Fist
```

15.4. Update Debug Variables

```
messageCount++;
```

This increases the message counter by one.

It means one more valid message has been received.

```
lastMessageTime = millis();
```

This records the current time. It is used for the green indicator dot.

```
println("Received: " + lastRawValue + " -> class " + currentClass + " -> " +
    currentGesture);
```

This prints the received value, class number, and gesture label to the console.

For example:

```
Received: 1.0 -> class 1 -> Open Hand
```

```
}
```

This closes the `oscEvent()` function.

16. The `classToGesture()` Function

This function converts a class number into a text label.

```
String classToGesture(int c) {
```

This defines a function that receives an integer class number and returns a string.

```
switch(c) {
```

This starts a switch statement. It checks the value of `c`.

```
case 1:
```

This case runs when `c` is 1.

```
return class1Gesture;
```

This returns the Class 1 gesture label, currently `Open Hand`.

```
case 2:
```

This case runs when `c` is 2.

```
return class2Gesture;
```

This returns the Class 2 gesture label, currently `Closed Fist`.

```
default:
```

This runs when the class is neither 1 nor 2.

```
return "Unknown Class";
```

This returns `Unknown Class` for unexpected values.

```
}
```

This closes the switch statement.

```
}
```

This closes the function.

17. The `keyPressed()` Function

This function provides an optional reset feature.

```
void keyPressed() {
```

This function runs whenever the user presses a key.

```
if (key == 'r' || key == 'R') {
```

This checks whether the pressed key is lowercase `r` or uppercase `R`.

The symbol `||` means "or".

So the condition means:

```
If the user pressed r or R
```

```
currentClass = 0;
```

This resets the class number to 0.

```
currentGesture = "Waiting for Wekinator...";
```

This resets the displayed gesture text to the waiting message.

```
lastRawValue = 0.0;
```

This resets the last raw Wekinator value.

```
println("Reset output display.");
```

This prints a reset message to the console.

```
}
```

This closes the key condition.

```
}
```

This closes the `keyPressed()` function.

18. Complete Program Flow

The full execution flow is:

```
1. Processing starts the sketch.
2. setup() runs once.
3. The window is created.
4. oscP5 starts listening on port 12000.
5. draw() begins looping.
6. The sketch shows a waiting screen.
7. Wekinator sends an OSC message to /wek/outputs.
8. oscEvent() is automatically called.
9. The sketch checks the OSC address.
10. The sketch reads the first output value.
11. The value is rounded to a class number.
12. The class number is converted into a gesture label.
13. The message counter and timestamp are updated.
14. draw() displays the updated class and gesture label.
15. The process repeats in real time.
```

19. Correct Running Order

For the complete project to work, run the components in this order.

19.1. During Training

During recording and training, the output sketch is optional.

Recommended order:

```
1. Run the input Processing sketch.
2. Open Wekinator.
3. Start listening in Wekinator.
4. Record Class 1 examples.
5. Record Class 2 examples.
6. Train the classifier.
```

19.2. During Testing and Demonstration

During testing, all three components should run:

```
1. Run the input Processing sketch.
2. Open the trained Wekinator project.
3. Run this output Processing sketch.
4. Click Run in Wekinator.
5. Show gestures to the camera.
6. Watch the output sketch display the prediction.
```

20. Example Use Case

Assume the project uses two classes:

```
Class 1 → Open Hand
Class 2 → Closed Fist
```

If the user shows an open hand to the input sketch:

```
Input sketch sends 2500 grayscale pixel values
        ↓
Wekinator predicts Class 1
        ↓
Wekinator sends /wek/outputs 1.0 to port 12000
        ↓
Output sketch receives 1.0
        ↓
Output sketch displays Open Hand
```

If the user shows a closed fist:

```
Input sketch sends 2500 grayscale pixel values
        ↓
Wekinator predicts Class 2
        ↓
Wekinator sends /wek/outputs 2.0 to port 12000
        ↓
Output sketch receives 2.0
        ↓
Output sketch displays Closed Fist
```

21. How This Code Fulfills the Exercise 2 Requirements

Exercise 2 requires a second Processing sketch that receives Wekinator's predicted gesture class and displays a text label on screen.

This code fulfills that requirement as follows:

Exercise Requirement	How This Code Fulfills It
Receive OSC messages from Wekinator	Uses `oscP5 = new OscP5(this, inPort)`
Listen on output port	Uses `inPort = 12000`
Match Wekinator output address	Uses `oscAddress = "/wek/outputs"`
Read predicted gesture class	Uses `lastRawValue = msg.get(0).floatValue()`
Convert classifier output to class number	Uses `currentClass = round(lastRawValue)`
Display text label	Uses `currentGesture` and `text(currentGesture, ...)`
Support two gesture classes	Uses `class1Gesture` and `class2Gesture`
Give real-time feedback	Updates continuously in `draw()`
Provide debugging information	Shows message count, raw value, port, and address
Handle unexpected messages	Ignores wrong OSC addresses and empty messages

Therefore, this sketch correctly implements the visualization/output part of the gesture classification exercise.

22. Strengths of This Output Sketch

22.1. Clear Visualization

The sketch clearly displays:

```
Received gesture class
Recognized gesture name
```

This makes it easy to demonstrate the system.

22.2. Simple Class Mapping

The class mapping is easy to edit:

```
String class1Gesture = "Open Hand";
String class2Gesture = "Closed Fist";
```

If the gestures change, only these two lines need to be changed.

22.3. Correct OSC Setup

The sketch listens on the correct Wekinator output port:

```
12000
```

and expects the correct Wekinator output address:

```
/wek/outputs
```

22.4. Debugging Support

The debug panel helps detect communication problems.

If messages are being received, the message count increases and the indicator dot flashes green.

22.5. Safe Message Handling

The sketch checks the OSC address and checks whether the message contains a value before using it. This makes the code safer and easier to debug.

23. Limitations of This Sketch

23.1. It Depends on Wekinator

This sketch does not classify anything by itself. It only displays the result that Wekinator sends.

If Wekinator is not running or not in Run mode, this sketch will stay in the waiting state.

23.2. It Only Handles Two Named Classes by Default

The sketch is currently configured for two classes:

```
Open Hand
Closed Fist
```

It can be extended to more classes, but the `classToGesture()` function would need more cases.

23.3. It Does Not Show Camera Input

This is intentional. The input sketch handles the camera. This output sketch only visualizes the prediction.

23.4. It Cannot Fix Bad Training

If Wekinator predicts incorrectly because the training data is bad, this sketch will still display the wrong class. The output sketch only displays the prediction; it does not improve the model.

24. Debugging Checklist

24.1. If the Output Sketch Stays on "Waiting for Wekinator..."

Check the following:

- Is Wekinator open?

- Is the Wekinator project trained?
- Did you click **Run** in Wekinator?
- Is Wekinator output port set to `12000`?
- Is Wekinator output message set to `/wek/outputs`?
- Is the output sketch running?
- Is another program already using port `12000`?

24.2. If Message Count Stays at 0

This means the output sketch is not receiving valid `/wek/outputs` messages.

Possible causes:

- Wrong Wekinator output port
- Wrong Wekinator output OSC address
- Wekinator not in Run mode
- Output sketch started on a different port
- Firewall or network issue, although localhost usually works

24.3. If the Message Count Increases but the Gesture Is Wrong

This means OSC communication is working, but Wekinator is predicting the wrong class.

Possible causes:

- Bad training examples
- Hand was outside the crop box during training
- Class 1 and Class 2 examples were mixed accidentally
- Lighting changed between training and testing
- KNN value or classifier settings are wrong
- Wekinator was not retrained after changing settings

24.4. If the Raw Value Is Not 1.0 or 2.0

Possible causes:

- Wekinator output type is not set to classification
- Number of outputs is not set to 1
- The wrong Wekinator project is open
- Wekinator is sending continuous outputs instead of classifier outputs

Correct Wekinator output type should be:

```
All classifiers
```

with:

```
1 output
2 classes
```

25. How to Extend This Code for More Classes

If the project has three gestures, for example:

```
Class 1 → Open Hand
Class 2 → Closed Fist
Class 3 → Thumbs Up
```

Add a new label:

```
String class3Gesture = "Thumbs Up";
```

Then add a new case:

```
case 3:
  return class3Gesture;
```

The updated function would be:

```
String classToGesture(int c) {
  switch(c) {
    case 1:
      return class1Gesture;
    case 2:
      return class2Gesture;
    case 3:
      return class3Gesture;
    default:
      return "Unknown Class";
  }
}
```

Wekinator would also need to be configured with three classes.

26. Explanation of OSC in This Sketch

OSC stands for **Open Sound Control**. It is a communication protocol often used in creative coding, music, interactive systems, and multimedia applications.

In this project, OSC connects Wekinator and Processing.

The input sketch sends:

```
/wek/inputs
```

to Wekinator.

Wekinator sends:

```
/wek/outputs
```

to this output sketch.

This output sketch listens for messages that look conceptually like this:

```
/wek/outputs 1.0
```

or:

```
/wek/outputs 2.0
```

The first part is the OSC address. The second part is the predicted class value.

27. Explanation of the Classifier Output

Wekinator's classification output is represented as a number.

For two classes:

```
1.0 = Class 1
2.0 = Class 2
```

The output sketch uses:

```
currentClass = round(lastRawValue);
```

to convert Wekinator's value into an integer.

Then it uses:

```
currentGesture = classToGesture(currentClass);
```

to convert the class number into a human-readable label.

This is why the output sketch can display `Open Hand` instead of only showing `1.0`.

28. Why This Output Is Useful for the Project

The project is not only about training a classifier. It is about building an interactive system.

An interactive system needs feedback.

Without this output sketch, Wekinator may classify the gesture internally, but the user would not have a clear visual response.

This sketch gives immediate feedback:

```
The user performs a gesture
        ↓
The classifier predicts a class
```

```
        ↓
The screen displays the gesture name
```

That makes the system understandable and demonstrable.

29. Recommended Final Project Description for This Sketch

A good final report paragraph could say:

```
The output Processing sketch receives the classification result from Wekinator using
    OSC. It listens on port 12000 for messages with the address /wek/outputs. Since
    the Wekinator model is configured as a two-class classifier with one output, the
    sketch expects a single numerical value: 1.0 for Class 1 and 2.0 for Class 2. The
     received value is rounded to an integer class label and mapped to a readable
    gesture name, such as Open Hand or Closed Fist. The sketch then displays the
    class number and gesture label on screen while also changing the background color
     and showing debugging information. This provides real-time visual feedback for
    the gesture classification system.
```

30. Final Conclusion

The `GestureOutput.pde` sketch is a correct and useful output sketch for Exercise 2.

It fulfills the visualization requirement by receiving Wekinator's predicted class through OSC and displaying the corresponding gesture label on the Processing screen.

The current configuration is:

```
OSC input port: 12000
Expected OSC message: /wek/outputs
Number of Wekinator outputs: 1
Output type: Classification
Number of classes: 2
Class 1 label: Open Hand
Class 2 label: Closed Fist
```

This output sketch completes the full interactive machine learning pipeline:

```
Gesture shown to webcam
        ↓
Input sketch extracts image features
        ↓
Wekinator classifies the gesture using KNN
        ↓
Output sketch receives /wek/outputs
        ↓
Recognized gesture is displayed on screen
```

The sketch is simple, clear, and reliable. It also includes helpful debugging tools such as a message counter, raw value display, OSC configuration display, and a flashing indicator dot when messages are received.

Together with the input sketch and Wekinator project, this code supports the main goal of Exercise 2: building a real-time gesture classification system using Processing, OSC, and Wekinator.